```
--¬Vincent MORIN¬Universite de Bretagne Occidentale¬janvier 2025¬Licence CC BY-SA 4.0¶--¬1¬2¬3¬4¬5¬6¬7¬8¬9¬10¬11¬12¶¶¶separate(¬IDL )¶¬¬¬¬¬-----------
-¶procedure¬¬¬¬¬PRETTY_DIANA¬¬( OPTION :CHARACTER¬:= 'U' )¶¬¬¬¬¬-------------------¶is¶  OFILE¬¬: FILE_TYPE;¬¬¬¬¬¬¬¬¬¬--| LE FICHIER DE SORTIE IMPRESSION¶¶  D
EBUG_PRETTY¬: BOOLEAN¬¬:= FALSE;¶¶¶¬¬¬¬------------¶  procedure¬¬¬IMPRIME¶  is¬¬¬¬¬¬¬¬¬¬----¶¶    LAST_PAGE¬¬¬: VPG_IDX¬:= VPG_IDX( DI( XD_HIGH_PAGE,¬TREE_ROO
T¬) );¶¶    type STATUS¬¬¬is ( PRINT, PRINT_AS, NO_PRINT );¬¬¬¬¬¬--| MARQUAGE DE LIGNE DE PAGE¬: IMPRIMER, IMPRIMER COMME ALL_SOURCE, LAISSER¶    type
PRINT_STATUS_VECTOR¬¬is array(¬LINE_IDX ) of STATUS;¬¬¬¬¬¬--| VECTEUR MARQUAGE D UNE PAGE¶    type PRINT_STATUS_ARRAY¬¬is array(¬1 .. LAST_PAGE ) of¬PR
INT_STATUS_VECTOR;¬¬¬--| MARQUAGE DE TOUS LES TREES DU FICHIER ARBRE (INDICAGE PAR PAGE ET¬LIGNE)¶¶    PRINT_STATUS¬¬¬: PRINT_STATUS_ARRAY¬:= (others
=> (others => PRINT) );¶    USER_ROOT¬¬¬: TREE¬¬¬:= D( XD_USER_ROOT,¬TREE_ROOT¬);¶    COMPLTN_STRUCT¬¬¬: TREE¬¬¬:= D( XD_STRUCTURE,¬USER_ROOT¬);¶¶¶¶¬¬
¬¬------¶    procedure¬¬¬INDENT¬¬¬( IND :INTEGER )¶¬¬¬¬¬-----¶    is¶      I¬¬: INTEGER¬¬:= IND;¶    begin¶      NEW_LINE;¬¬¬¬¬¬¬¬¬¬¬¬¬--| SE FAIT SUR U
NE¬NOUVELLE LIGNE¶      while I >= 8 loop¶        PUT( "       | " );¬¬¬¬¬¬¬¬¬¬¬¬¬--| UNE SUITE DE 8 CARACTERES¬AVEC UN |¬A 7¶        I¬:= I - 8;¬¬¬¬¬¬¬¬¬
¬¬--| 8 DE MOINS¶      end¬loop;¶      for¬N in 1 ..¬I loop PUT( ' ' ); end loop;¬¬¬¬¬¬¬¬¬--| RELIQUAT DE BLANCS¶¶    end¬INDENT;¶¬------¶¶¶¬¬¬¬¬-------
----¶    procedure¬¬¬MARK_STRUCT¬¬( T :TREE¬)¬¬¬¬¬--| MARQUE EN PRINT_AS LES TREES DE LA CATEGORIE "ALL_SOURCE"¶¬¬¬¬-------------¶    is¶    begin¶
if¬T = TREE_VOID  or else  T = TREE_NIL  or else  T = TREE_VIRGIN¬¬¬¬¬¬--| TREE VERS UN NOEUD SANS ATTRIBUT¶¬or else  T.PT = HI¬¬¬¬¬¬¬¬¬¬¬--| TREE REPRE
SENTANT UN ENTIER 16 BITS¬NEGATIF OU UNE POSITION SOURCE¶¬or else  ( (T.PT=P or T.PT=L)¬ and then¬ PRINT_STATUS( T.PG¬)( T.LN )¬/= PRINT )¬¬¬--| DEJA
MARQUE¶      then¶         return;¬¬¬¬¬¬¬¬¬¬¬¬--| NE RIEN FAIRE¶      end¬if;¶¶       if¬(T.PT = P¬ or  T.PT¬= L) and then  T.TY¬in CLASS_ALL_SOURCE¬¬¬¬¬
¬-- POINTEUR OU ATTRIBUT LISTE¬DANS LA CLASSE ELEMENTS DE CODE SOURCE¶      then¶         PRINT_STATUS( T.PG )(¬T.LN ) :=¬PRINT_AS;¬¬¬¬¬¬¬¬¬--| INITIALI
SER EN MARQUER EN¬A IMPRIMER ELEMENT "AS_"¶¶        if  ARITY( T ) = ARBITRARY  then¬¬¬¬¬¬¬¬¬--| ARITE¬QUELCONQUE : LISTE¶¬declare¶¬  ITEM_LIST¬: SEQ_T
YPE¬:= LIST( T );¬¬¬¬¬¬¬--| AMENER LA LISTE¬POINTEE¶¬  ITEM¬¬: TREE;¶¬begin¶¬  while  not IS_EMPTY( ITEM_LIST )  loop¬¬¬¬¬¬¬¬--| TANT QUE PAS VIDE¶¬
POP( ITEM_LIST,¬ITEM );¬¬¬¬¬¬¬¬¬¬--| EXTRAIRE UN POINTEUR DE LA LISTE¶¬    MARK_STRUCT( ITEM );¬¬¬¬¬¬¬¬¬¬--| FAIRE¬LE MARQUAGE EN SUIVANT CE POINTEUR¶¬
 end loop;¶¬end;¶¶       else¬¬¬¬¬¬¬¬¬¬¬¬--| ARITE¬DEFINIE¶¬for  I in¬1 .. ATTR_NBR( ARITIES'POS( ARITY( T ) ) )  loop¬¬¬¬¬¬¬--| POUR TOUS LES CHAMPS¶¬
  MARK_STRUCT( DABS( I, T ) );¬¬¬¬¬¬¬¬¬--| FAIRE¬LE MARQUAGE EN SUIVANT LE POINTEUR DU CHAMP¶¬end loop;¶       end if;¶      end¬if;¶¶    end¬MARK_STR
UCT;¶¬-----------¶¶¶¬¬¬¬--------------¶    procedure¬¬¬PRINT_DIANA¬¬( T :TREE; IND :NATURAL; PARENT :TREE )¶¬¬¬¬-------------¶    is¶      A_SUB¬: INTEGER;
¶    begin¶      if¬DEBUG_PRETTY  then PUT_LINE( "PRINT_DIANA" ); end¬if;¶¶--.................................................................
........................¶¶--¬¬IMPRESSION AU BESOIN DU POINTEUR DE NOEUD¶¶--.....................................................................
........................¶      PRINT_NOD.PRINT_TREE( T¬);¬¬¬¬¬¬¬¬¬¬--| IMPRIMER LE POINTEUR D'ARBRE¶¶      if¬T = TREE_VOID  or else  T = TREE_NIL  or
else  T = TREE_VIRGIN¶¬or else  T.PT = HI¬or else T.PT = S¶      then return;¬¬¬¬¬¬¬¬¬¬¬¬--| FINI¶      end¬if;¶¶      A_SUB := N_SPEC( T.TY ).NS_FIRST
_A;¶      if¬T.TY in CLASS_ALL_SOURCE  then¬¬¬¬¬¬¬¬¬--| NOEUD¬"ALL_SOURCE"¶¶¬¬¬¬------------------------¶¬¬¬¬IMPRIME_POSITION_SOURCE:¶        declare¶¬S
POS¬: TREE¬:= D( LX_SRCPOS, T );¬¬¬¬¬¬--| POSITION SOURCE¬: (LIGNE,COL)¶        begin¶¬PUT( " SLOC(" );¶¬if  SPOS.PT = S  and then  SPOS.COL in 1 .. 25
4  then¶¬  declare¶¬    IML :¬constant STRING¬:= INTEGER'IMAGE( DI( XD_NUMBER, GET_SOURCE_LINE(¬SPOS ) ) );¶¬    IMC :¬constant STRING¬:= SRCCOL_IDX'I
MAGE( GET_SOURCE_COL( SPOS ) );¶¬  begin¶¬    PUT( IML( 2 .. IML'LENGTH¬) & "," &¬IMC( 2 ..¬IMC'LENGTH ) );¶¬  end;¶¬else¶¬  PRINT_TREE( SPOS );¶¬end
if;¶¬PUT( ')' );¶        end¬IMPRIME_POSITION_SOURCE;¶¬¬-----------------------¶¶¶        if  T.TY in¬CLASS_SOURCE_NAME  then¬¬¬¬¬¬¬¬¬¬--| CHOSES QUI ON
T UN SYMBOLE¬ASSOCIE¶¬¬------------------¶¬¬¬NODE_WITH_IDENTIFER:¶¬declare¶¬  SYMREP¬¬: TREE¬¬:= D( LX_SYMREP, T );¶¬  DEFLIST¬¬: SEQ_TYPE;¶¬  DEF¬¬
: TREE;¶¬begin¶¬  if  SYMREP.TY = DN_SYMBOL_REP  then¶¬    DEFLIST := LIST( SYMREP );¶¬    while¬ not IS_EMPTY( DEFLIST )  loop¶¬      POP( DEFLIST,¬D
EF );¶¬      if¬DEF.TY = DN_DEF  and then  D(¬XD_SOURCE_NAME, DEF¬) = T  then¶¬        PUT( ' ' );¬PRINT_TREE( DEF );¶¬        PUT( ' ' );¬PRINT_TREE(
 D( XD_REGION_DEF,¬DEF ) );¶¬        exit;¶¬      end¬if;¶¬    end loop;¶¬  end if;¶¬end¬NODE_WITH_IDENTIFER;¶¬¬----------------¶        end if;¶¶
    end¬if;¶--.......................................................................................¶¶--¬¬IMPRESSION EVENTUELLE DU CORPS
DE NOEUD¶¶--.......................................................................................¶      if¬PRINT_STATUS( T.PG )( T.LN ) =
 NO_PRINT¬¬¬¬¬¬¬¬¬--| PAS D'IMPRESSION DU NOEUD, RETOUR¶      then return;¶      end¬if;¶¶      PRINT_STATUS(¬T.PG )( T.LN ) := NO_PRINT;¬¬¬¬¬¬¬¬¬--| ME
NTIONNER NE PLUS IMPRIMER A L AVENIR¶¶¶      if¬T.TY = DN_COMPILATION_UNIT  then¬¬¬¬¬¬¬¬¬¬--| CAS SPECIAL DE L'UNITE DE¬COMPILATION¶¬¬¬¬¬-------------
-------------¶¬¬¬¬¬COMPILATION_UNIT_SPECIAL_CASE:¶        declare¶¬TRANS_WITH_LIST¬: SEQ_TYPE¬:= LIST( T );¬¬¬¬¬¬¬--| LISTE¬DES WITH¶¬TRANS_WITH¬: TRE
E;¶¬COMP_UNIT¬¬: TREE;¶        begin¶¬if  TRANS_WITH_LIST.FIRST /= TREE_VIRGIN  then¬¬¬¬¬¬¬¬--| TETE DE LISTE NON INITIALISEE : LIB_PHASE NON¬FAITE¶¶¬
 if DEBUG_PRETTY then PUT_LINE( "dn_compilation_unit trans_with_list" ); end if;¶¶¬  while  not IS_EMPTY( TRANS_WITH_LIST )  loop¶¬    POP( TRANS_WITH
_LIST, TRANS_WITH );¬¬¬¬¬¬¬¬¬--| EXTRAIRE UN ELEMENT DE LISTE WITH¶¶¬    COMP_UNIT := D(¬TW_COMP_UNIT, TRANS_WITH );¶¬    if D(¬XD_NBR_PAGES, COMP_UNIT
 ) /= TREE_VIRGIN then¶¬      for¬ I in COMP_UNIT.PG .. COMP_UNIT.PG + PAGE_IDX( DI( XD_NBR_PAGES, COMP_UNIT ) ) - 1  loop¶¬        declare¶¬¬PS : STA
TUS := NO_PRINT;¶¬        begin¶¬¬if  OPTION = 'P'  then PS := PRINT_AS; elsif  OPTION = 'A'¬then PS := PRINT; end if;¶¬¬PRINT_STATUS( I ) := (others
=> PS);¬¬¬¬¬¬¬¬--| IMPRIMER SUIVANT OPTION¶¬        end;¶¬      end¬loop;¶¬    end if;¶¬  end loop;¶¶¬end if;¶        end¬COMPILATION_UNIT_SPECIAL_CASE
;¶¬¬---------------------------¶      end¬if;¶¶¬¬¬¬¬--------------------------¶¬¬¬¬¬TRAITER_LES_DESCENDANTS:¶      declare¶        NB_STRUC_CHILD¬: ATT
R_NBR¬:= 0;¶¶¬¬¬---------------------¶        procedure¬¬PRINT_NON_STRUCT_ATTR¬( A_SUB :INTEGER; T¬:TREE; IND :INTEGER; PARENT :TREE)¬--| IMPRESSION S
ANS¬DETAIL DES SOUS ARBRES (PRINT_TREE)¶¬¬¬---------------------¶        is¶        begin¶¬if  DEBUG_PRETTY  then PUT_LINE( "print_non_struct_attr" );
¬end if;¶¶¬if  T = TREE_VOID  or else  T¬= TREE_VIRGIN  then¶--¬  PUT( "^" & NODE_NAME'IMAGE(¬T.TY ) );¶¶--PUT( " " & ATTRIBUTE_NAME'IMAGE( A_SPEC( A_
SUB ).ATTR ) );¶return;¶¶¬end if;¶¬if  T.PT = S  then PUT( "c" &¬SRCCOL_IDX'IMAGE( T.COL ) ); return; end if;¶¬if  T.PT = HI then PUT( "#" &¬NODE_NAME

'IMAGE( T.NOTY ) ); return; end if;¶¶¬INDENT( IND );¶¬PUT( ATTRIBUTE_NAME'IMAGE( A_SPEC( A_SUB ).ATTR )¬);¶¶¬if  not A_SPEC( A_SUB ).IS_LIST  and then
  T.TY /= DN_LIST¬then¬¬¬¬¬¬--| PAS UNE LISTE¶¬  PUT(": ");¶¬  if  T.PG > 0  and¬then  T.TY = DN_REAL_VAL  then¶¬    PRINT_TREE( D( XD_NUMER,T¬) ); PUT
(¬'/' ); PRINT_TREE( D( XD_DENOM, T ) );¶¬  else¶¬    PRINT_TREE( T );¶¬  end if;¶¶¬  if  T.PG > 0  and¬then  T.TY = DN_SYMBOL_REP  then¶¬    PUT( ' '
 & PRINT_NAME( T ) );¶¬  end if;¶¶¬else¬¬¬---------------------¬¬¬¬¬¬--| UNE LISTE¶¬¬¬¬IMPRIME_UNE_LISTE:¶¬  declare¶¬    SQ¬:  SEQ_TYPE¬:= ( FIRST=> T,
NEXT=> TREE_NIL );¶¬    E¬: TREE;¶¬  begin¶¬    PUT( ": { " );¶¬    while¬ not IS_EMPTY( SQ )¬ loop¶¬      POP( SQ, E );¬INDENT( IND ); PRINT_TREE( E
);¶¬    end loop;¶¬    PUT( " }" );¶¶¬  end¬IMPRIME_UNE_LISTE;¶¬¬-------------¶¬end if;¶        end¬PRINT_NON_STRUCT_ATTR;¶¬¬-----------------------
¶¶¶¬¬¬¬--------------------¶        procedure¬¬¬PRINT_STRUCT_ATTR¬¬( A_SUB :¬INTEGER; T : TREE; IND :INTEGER; PARENT¬: TREE )¬--| IMPRESSION AVEC¬DETAIL
DES SOUS ARBRES (PRINT_DIANA)¶¬¬¬¬------------------¶        is¶¬ATNBR¬: ATTRIBUTE_NAME¬:= A_SPEC( A_SUB ).ATTR;¶        begin¶¶¬if  T = TREE_VOID  or
else  T¬= TREE_VIRGIN  then¶--¬  PUT( "^" & NODE_NAME'IMAGE(¬T.TY ) );¶--PUT( " " & ATTRIBUTE_NAME'IMAGE( ATNBR ) );¶return;¶¬end if;¶¶¬if  T.PT = S
then PUT( "c" &¬SRCCOL_IDX'IMAGE( T.COL ) ); return; end if;¶¬if  T.PT = HI then PUT( "#" &¬NODE_NAME'IMAGE( T.NOTY ) ); return; end if;¶¶¬if  DEBUG_P
RETTY  then PUT_LINE( "print_struct_attr" ); end¬if;¶¶¬if  T.PT = P  and then  T.TY in CLASS_STANDARD_IDL  and then  not A_SPEC( A_SUB¬).IS_LIST¶¬then
¶¬  PRINT_NON_STRUCT_ATTR( A_SUB, T, IND,¬PARENT );¶¬  return;¶¬elsif  T.PT = P  and then  T.TY = DN_REAL_VAL  then¶¬  PRINT_NON_STRUCT_ATTR( A_SUB, T
, IND,¬PARENT );¶¬  return;¶¬end if;¶¶¬INDENT( IND );¶¬PUT( ATTRIBUTE_NAME'IMAGE( ATNBR ) );¶¶¬if  not A_SPEC( A_SUB ).IS_LIST  then¬¬¬¬¬¬¬¬¬--| PAS UN
E LISTE¶¬  if  T.PT = S  then¶¬    PUT( ": " ); PRINT_TREE( T );¶¬  else¶¬    PUT( ": ") ; PRINT_DIANA(¬T, IND+2,¬PARENT );¶¬  end if;¶¶¬else¶¬¬¬¬¬---
-------------------¶¬¬¬¬¬IMPRIME_LISTE_DETAILLEE:¬¬¬¬¬¬--| UNE LISTE¶¬  declare¶¬    SQ¬: SEQ_TYPE¬:= ( FIRST=> T, NEXT=> TREE_NIL );¶¬  begin¶¬    if
  IS_EMPTY( SQ )  then PUT( ": {}" );¬¬¬¬¬¬¬¬--| LISTE¬VIDE¶¬    else¶¬¬¬¬--------------¶¬¬¬NON_EMPTY_LIST:¶¬      declare¶¬        HD¬: TREE¬:= HEAD( S
Q );¶¬      begin¶¬        if  IS_EMPTY( TAIL( SQ ) )  then¬¬¬¬¬¬¬¬--| SINGLETON¶¬¬PUT( ": { " ); PRINT_DIANA( HD, IND+2, PARENT ); PUT( " }" );¶¬
   else¬¬¬¬¬¬¬¬¬¬¬--| LISTE¬GENERALE¶¬¬PUT( ':' );¶¬¬INDENT( IND );¶¬¬PUT( "{ "¬);¶¬¬PRINT_DIANA( HD, IND+4, PARENT );¶¬¬SQ := TAIL( SQ );¶¬¬while  not
 IS_EMPTY( SQ )  loop¶¬¬  INDENT(¬IND + 2 );¶¬¬  PRINT_DIANA( HEAD( SQ ), IND+4, PARENT );¶¬¬  SQ := TAIL( SQ );¶¬¬end loop;¶¬¬PUT( " }"¬);¶¬        e
nd if;¶¬      end¬NON_EMPTY_LIST;¶¬¬-------------¶¶¬    end if;¶¬  end¬¬IMPRIME_LISTE_DETAILLEE;¶¬¬¬--------------------¶¬end if;¶¶        end¬PRI
NT_STRUCT_ATTR;¶¬¬-----------------¶¶¶¬¬¬--------------------¶        procedure¬¬¬MAYBE_NON_STRUCT_ATTR¬¬( A_SUB :¬INTEGER; T : TREE;¶¬¬¬¬¬¬¬¬  IND
  :¬INTEGER; PARENT, GRAND_PARENT¬: TREE )¶        is¶        begin¶¬if  T.PT = S  or  T.PT = HI  then return; end if;¶¶¬if  DEBUG_PRETTY  then¶¬  PUT
( "maybe_non_struct_attr (" & "P" & VPG_IDX'IMAGE( T.PG ) & " L"¬& LINE_IDX'IMAGE( T.LN ) & ") " );¶¬  if  T.PG /= 0  then PUT( " status=" &¬STATUS'IM
AGE( PRINT_STATUS( T.PG )( T.LN ) ) ); end if;¶¬  NEW_LINE;¶¬end if;¶¶¬if  T.PG = 0  or else  PRINT_STATUS( T.PG )( T.LN¬) = NO_PRINT  then¶¬  PRINT_N
ON_STRUCT_ATTR ( A_SUB, T, IND, PARENT );¶¬else¶¬  PRINT_STRUCT_ATTR( A_SUB, T, IND, PARENT );¶¬end if;¶¶        end¬MAYBE_NON_STRUCT_ATTR;¶¬¬--------
------------¶¶¶¬¬¬------------------------¶        procedure¬¬¬PRINT_IF_NOT_STRUCTURAL ( A_SUB : INTEGER; T : TREE; IND : INTEGER; PARENT, GRAND_PARE
NT : TREE )¶¬¬¬¬-------------------¶        is¶        begin¶¶¬if  T.PT = HI  then¶¬  INDENT(¬IND );¶¬  PUT( ATTRIBUTE_NAME'IMAGE( A_SPEC( A_SUB )
.ATTR¬) );¶¬  PUT( ": HI noty="¬& NODE_NAME'IMAGE( T.NOTY ) &¬" abss=" & POSITIVE_SHORT'IMAGE( T.ABSS¬) & " nsiz=" & ATTR_NBR'IMAGE( T.NSIZ )¬);¶¬  re
turn;¶¬end if;¶¶¬if  T.PT = S  or else  T.PG =¬0  then¬¬¬¬¬¬¬¬¬¬--| S NIL¬VOID VIRGIN¶¬  return;¶¬end if;¶¶¬if  DEBUG_PRETTY  then PUT_LINE( "print_if_n
ot_structural" ); end if;¶¶¬if  PRINT_STATUS( T.PG )( T.LN ) /= PRINT  then¶¬  PRINT_NON_STRUCT_ATTR( A_SUB, T, IND,¬PARENT );¶¬else¶¬  MAYBE_NON_STRU
CT_ATTR( A_SUB, T, IND,¬PARENT, GRAND_PARENT );¶¬end if;¶¶        end¬PRINT_IF_NOT_STRUCTURAL;¶¬¬----------------------¶¶        begin¶        if  DEBU
G_PRETTY  then PUT_LINE( "traiter_les_descendants" );¬end if;¶¶        if  (T.PT =¬P  or  T.PT = L)  and then  T.TY in CLASS_ALL_SOURCE  then¶¬if  ARI
TY( T ) = ARBITRARY  then¬¬¬¬¬¬¬¬¬--| UN ELEMENT DE LISTE¶¬  NB_STRUC_CHILD :=¬1;¬¬¬¬¬¬¬¬¬¬--| UN SEUL ELEMENT¬STRUCTUREL D'ARBRE SYNTAXIQUE¶¬else¶¬  NB
_STRUC_CHILD :=¬ARITIES'POS( ARITY(¬T ) );¬¬¬¬¬¬¬¬--| AUTANT D'ELEMENTS STRUCTURELS QUE L'ARITE L'INDIQUE¶¬end if;¶        end if;¶¶        for  I in 1
¬.. NB_STRUC_CHILD  loop¬¬¬¬¬¬¬¬¬--| S'OCCUPER DES DESCENDANTS¬DE STRUCTURE SYNTAXIQUE (EVENTUELLEMENT)¶¬if  A_SPEC( A_SUB +¬INTEGER(I) - 1 ).ATTR /= L
X_SRCPOS  then¬¬¬¬¬¬--| SI PAS UN SRCPOS¶¬  MAYBE_NON_STRUCT_ATTR( A_SUB + INTEGER( I ) - 1, DABS ( I, T ), IND, T, PARENT );¶¬end if;¶        end loop
;¶¶        for  I in NB_STRUC_CHILD + 1 ..¬N_SPEC( T.TY ).NS_SIZE  loop¬¬¬¬¬¬¬--| APRES¬LES CHAMPS COMPRIS DANS L'ARITE SYNTAXIQUE D'AUTRE CHAMPS EVENT
UELS¶¬if  A_SPEC( A_SUB +¬INTEGER( I ) - 1 ).ATTR /= LX_SRCPOS  then¬¬¬¬¬¬--| SI PAS UN SRCPOS¶¬  PRINT_IF_NOT_STRUCTURAL( A_SUB + INTEGER( I ) -¬1, DA
BS (¬I, T ), IND, T, PARENT );¶¬end if;¶        end loop;¶¶        end¬¬¬TRAITER_LES_DESCENDANTS;¶¬¬----------------------¶¶        if¬DEBUG_PRETTY  the
n PUT_LINE( "print_diana ok" ); end if;¶¶    end¬PRINT_DIANA;¶¬----------¶¶  begin¶¶    PUT_LINE( "TREE_ROOT=" );¬   print_node( TREE_ROOT );¶    PUT
_LINE( "USER_ROOT=" );¬   print_node( USER_ROOT );¶¶    MARK_STRUCT( COMPLTN_STRUCT );¬¬¬¬¬¬¬¬--| MARQUER LES NOEUDS DE CLASS_ALL_SOURCE¶    PRINT_DIAN
A( COMPLTN_STRUCT, 0, TREE_VOID );¶    NEW_LINE;¶¶  end¬IMPRIME;¶¬-------¶¶begin¶  OPEN_IDL_TREE_FILE( IDL.LIB_PATH( 1..LIB_PATH_LENGTH ) & "$$$.TMP"
);¬¬¬--| OUVRIR LE FICHIER ARBRE TEMPORAIRE¶  CREATE(¬OFILE, OUT_FILE,"$$$_TREE.TXT" );¬¬¬¬¬¬¬--| CREER¬LE FICHIER IMPRESSION DE L'ARBRE¶  SET_OUTPUT(
OFILE¬);¬¬¬¬¬¬¬¬¬¬--| REDIRIGER LA SORTIE STANDARD VERS LE FICHIER IMPRESSION¶  IMPRIME;¬¬¬¬¬¬¬¬¬¬¬--| IMPRIMER L'ARBRE¶  SET_OUTPUT( STANDARD_OUTPUT¬);¬¬
¬¬¬¬¬--| REPOSITIONNER LA SORTIE STANDARD¶  CLOSE( OFILE );¬¬¬¬¬¬¬¬¬¬--| FERMER LE FICHIER IMPRESSION¶  CLOSE_IDL_TREE_FILE;¬¬¬¬¬¬¬¬¬--| FERMER LE FICHI
ER ARBRE¶¶exception¶  when others =>¬¬¬¬¬¬¬¬¬¬--| POUR TOUT PROBLEME¶    SET_OUTPUT( STANDARD_OUTPUT );¬¬¬¬¬¬¬¬--| REPOSITIONNER LA SORTIE STANDARD¶
CLOSE( OFILE );¬¬¬¬¬¬¬¬¬¬--| FERMER LE FICHIER IMPRESSION¶    CLOSE_IDL_TREE_FILE;¬¬¬¬¬¬¬¬¬--| FERMER LE FICHIER ARBRE¶    raise;¶end PRETTY_DIANA;¶
```